

Sesión 2: Algoritmos de Ordenamiento

Análisis comparativo de algoritmos iterativos y recursivos en C#

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

Agenda de Hoy

-  Importancia del ordenamiento eficiente
-  Algoritmos Cuadráticos: Burbuja, Inserción y Selección
-  Divide y Vencerás: Algoritmo QuickSort
-  Tiempos y complejidades comparativas en C#



De $O(n^2)$ a $O(n \log n)$

Cómo optimizar drásticamente la velocidad de ordenamiento estructural.

Algoritmos de Ordenamiento Cuadráticos

Los métodos tradicionales de comparación iterativa directa

1. Ordenamiento Burbuja (Bubble Sort)

Compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. El elemento más grande "flota" hacia el final en cada iteración.



9 es mayor que 2 → Se intercambian

Peor Caso

$O(n^2)$

Lista invertida por completo.

Mejor Caso

$O(n)$

Lista ya ordenada (con bandera de control).

Implementación: Bubble Sort en C#

```
public static void BubbleSort(int[] arr) { int n = arr.Length; for (int i = 0; i < n - 1; i++) { for (int j = 0; j < n - i - 1; j++) { if (arr[j] > arr[j + 1]) { int temp = arr[j]; // Intercambio (Swap) arr[j] = arr[j + 1]; arr[j + 1] = temp; } } } }
```


2. Ordenamiento por Inserción (Insertion Sort)

Divide el arreglo conceptualmente en una sección ordenada y una desordenada. Toma los elementos de la parte desordenada y los "inserta" en la posición correcta de la sección ordenada.



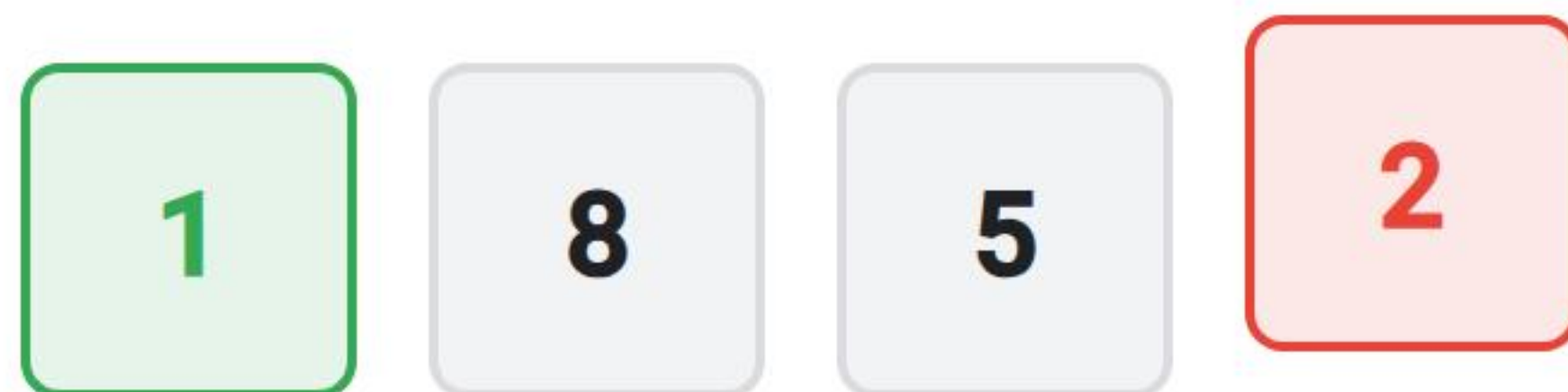
Ventaja clave: Es sumamente eficiente para conjuntos de datos que ya se encuentran parcialmente ordenados en escenarios del mundo real.

Implementación: Insertion Sort en C#

```
public static void InsertionSort(int[] arr) { int n = arr.Length; for (int i = 1; i < n; ++i) { int llave = arr[i]; int j = i - 1; while (j >= 0 && arr[j] > llave) { arr[j + 1] = arr[j]; // Desplazamiento j = j - 1; } arr[j + 1] = llave; } }
```


3. Ordenamiento por Selección (Selection Sort)

Busca repetidamente el elemento mínimo de la sección desordenada y lo coloca al principio. Reduce el número total de intercambios drásticamente a comparación de la Burbuja.



El 2 es el mínimo actual desordenado → Va al índice 1

⚠ **Nota de Eficiencia:** A diferencia de los anteriores, Selection Sort siempre realiza la misma cantidad de comparaciones, por lo que su mejor, peor y caso promedio son inamoviblemente $O(n^2)$.

Implementación: Selection Sort en C#

```
public static void SelectionSort(int[] arr) { int n = arr.Length; for (int i = 0; i < n - 1; i++) { int minIdx = i; for (int j = i + 1; j < n; j++) if (arr[j] < arr[minIdx]) minIdx = j; int temp = arr[minIdx]; // Swap al finalizar la búsqueda arr[minIdx] = arr[i]; arr[i] = temp; } }
```

Algoritmos Logarítmicos: Divide y Vencerás

Rompimiento recursivo del problema para optimizar el rendimiento asintótico

| El Paradigma "Divide y Vencerás"

Consiste en resolver un problema grande dividiéndolo de forma recursiva en subproblemas más pequeños del mismo tipo, hasta que estos sean lo suficientemente simples como para resolverse directamente.



Dividir

Fragmentar la estructura de datos mediante un punto estratégico (Pivote o Mitad).



Vencer

Ordenar las subdivisiones de forma recursiva independiente.



Combinar

Unir los subarreglos ordenados para reconstruir el set original completo.

4. Algoritmo QuickSort

Selecciona un elemento llamado **pivote** y particiona el arreglo a su alrededor, de modo que los elementos menores queden a la izquierda y los mayores a la derecha.

Flujo de Ejecución Recursivo:

```
[Menores que el Pivote] <-- PIVOTE --> [Mayores que el Pivote]
```

Al procesar los dos lados concurrentemente mediante recursión, reduce la complejidad del ordenamiento en promedio a un impresionante **$O(n \log n)$** .

Implementación: QuickSort en C# (Partición)

```
private static int Partition(int[] arr, int bajo, int alto) { int pivote = arr[alto]; // Pivote al final int i = (bajo - 1);  
for (int j = bajo; j < alto; j++) { if (arr[j] < pivote) { i++; int temp = arr[i]; // Swap arr[i] = arr[j]; arr[j] = temp; }  
} int temp2 = arr[i + 1]; arr[i + 1] = arr[alto]; arr[alto] = temp2; return i + 1; }
```


Implementación: QuickSort en C# (Recursión)

```
public static void QuickSort(int[] arr, int bajo, int alto) { if (bajo < alto) { // pi es el índice de
partición, arr[pi] ya está ordenado int pi = Partition(arr, bajo, alto); // Ordenar recursivamente los
subarreglos QuickSort(arr, bajo, pi - 1); QuickSort(arr, pi + 1, alto); } }
```


El Peor Caso de QuickSort

A pesar de ser increíblemente veloz, QuickSort sufre de una debilidad estructural asintótica según la selección del pivote:

Degeneración a $O(n^2)$:

Si el arreglo ya está ordenado (de forma ascendente o descendente) y elegimos siempre el último elemento como pivote, las particiones quedan totalmente desbalanceadas (tamaño 0 y $n-1$).

💡 **Solución Profesional:** Elegir un pivote de forma completamente aleatoria o mediante la técnica de la mediana de tres elementos.

Comparativa de Rendimiento Técnico

Análisis de las cotas superiores en memoria y procesamiento

Tabla Comparativa Asintótica

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Espacio (RAM)
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Rendimiento en el Mundo Real ($n=100,000$)

Para ordenar 100,000 registros numéricos en una computadora estándar:

 **Algoritmos $O(n^2)$:** Pueden demorar entre **15 a 30 segundos** de procesamiento puro.

 **Algoritmos $O(n \log n)$:** Resuelven la tarea en cuestión de **milisegundos (0.02 segundos)**.

 La optimización asintótica en el código es lo que permite el correcto escalamiento de los sistemas corporativos modernos.

Práctica en el Laboratorio

Conducción de pruebas de rendimiento reales bajo el SDK de .NET

Midiendo Tiempos con Stopwatch

La clase de diagnóstico nativa `System.Diagnostics.Stopwatch` permite calcular de manera empírica la velocidad exacta del algoritmo desarrollado en el entorno local.

```
using System.Diagnostics;

Stopwatch sw = new Stopwatch();
sw.Start(); // Arranca el reloj

BubbleSort(miArreglo);

sw.Stop(); // Detiene el reloj
Console.WriteLine($"Tiempo transcurrido: {sw.ElapsedMilliseconds} ms");
```


| Estabilidad en el Ordenamiento

Un algoritmo es **estable** si preserva el orden relativo original de los elementos que contienen llaves o valores idénticos.

Crucial cuando ordenas listas de objetos complejos (ej. Alumnos por apellidos, y luego por notas).

✓ Burbuja e Inserción: **Estables**

✗ Selección y QuickSort: **Inestables**

Valor de la Unidad: Honestidad



"La honestidad intelectual es un principio conductor de toda investigación y desarrollo de software profesional. Reconocer la autoría y construir código con integridad define tu identidad de ingeniero."

— Código de Ética de la Facultad de Ingeniería

Conclusiones

Compromiso

Los métodos $O(n^2)$ son sencillos de programar pero insostenibles a gran escala.

Recursividad

QuickSort demuestra que un cambio de paradigma lógico revoluciona el rendimiento.

Empirismo

El uso de Stopwatch consolida la relación entre la teoría Big O y la ejecución en hardware.

¿Dudas o Preguntas de la Sesión 2?

¡Muchas gracias por su atención!

 Fernando Vicente: 3021894750101@ingenieria.usac.edu.gt

 Canales del Foro Técnico en UEDI